

PAULT DOCUMENTATION

2026 02 15 Creation Window Variables Richmedia Design

Source: docs/plans/2026-02-15-creation-window-variables-richmedia-design.md

Theme: Clean Technical

Generated: February 26, 2026

Design: New Prompt Window, Auto-Expanding Variables, Rich Media

Date: 2026-02-15

Status: Approved

Overview

Three features to improve prompt creation and content richness in Pault:

1. **New Prompt Creation Window** — dedicated window for creating prompts (replaces inline creation)
2. **Auto-Expanding Variable Fields** — variable text fields grow with content instead of being fixed single-line
3. **Rich Media Attachments** — attach images, video, audio, and office files to prompts with inline image support

Feature 1: New Prompt Creation Window

Problem

Creating a prompt currently happens inline — Cmd+N inserts a blank prompt and selects it in the detail view. This mixes creation with editing and doesn't give users a focused creation experience.

Design

A separate `NSWindow` (SwiftUI window scene) opens when pressing Cmd+N, containing:

- Title field
- Content editor (rich text, same editor as detail view)
- Tag picker (reuses existing `TagPickerView`)
- Template variables section (auto-detected from `{{content}}`)
- Cancel and Create Prompt buttons

Window size: ~600x500, non-resizable or with min size constraints.

Behavior:

- "Create Prompt" inserts into `SwiftData`, closes window, and auto-selects in main sidebar
- "Cancel" discards and closes
- Communication between windows via `SwiftData` (shared model container) and `NotificationCenter`

Feature 2: Auto-Expanding Variable Fields

Problem

Variable fields are fixed-height `TextFields`. Users entering longer values (paragraphs, multi-line content) can't see what they've typed.

Design

Replace `TextField` with an auto-expanding text input:

- Starts as single-line height (like a standard form field)
- Grows vertically as text wraps or user enters newlines
- No max height — grows as needed
- Styled with rounded border to maintain form-like appearance

Implementation: `NSViewRepresentable` wrapping `NSTextView` with `isVerticallyResizable = true` and height tracking via layout manager notifications.

Feature 3: Rich Media Attachments

Data Model

New Attachment model:

- `id`: `UUID`
- `filename`: `String` — original filename
- `mediaType`: `String` — UTI type identifier
- `fileSize`: `Int64` — size in bytes
- `storageMode`: `String` — "embedded" or "referenced"
- `relativePath`: `String?` — path within app sandbox (embedded files)
- `bookmarkData`: `Data?` — security-scoped bookmark (referenced files)
- `thumbnailData`: `Data?` — cached thumbnail image data
- `sortOrder`: `Int` — display ordering
- `createdAt`: `Date`
- `prompt`: `Prompt?` — inverse relationship

Relationship: `Prompt (1) → (many) Attachment`, cascade delete.

Prompt model changes:

- Add `@Relationship(deleteRule: .cascade, inverse: \Attachment.prompt) var attachments: [Attachment]`
- Add `attributedContent`: `Data?` — RTF data for rich text content
- Keep `content`: `String` as plain text mirror (for search, templates, clipboard fallback)

Storage Strategy

- Files ≤ 10 MB $\rightarrow$ copied to `~/Library/Application Support/Pault/Attachments/<prompt-id>/<uuid>.<ext>`
- Files > 10 MB $\rightarrow$ stored as security-scoped bookmarks to original location
- On deletion: embedded files removed from disk, bookmarks discarded

Rich Text Editor

Replace `TextEditor(text:)` with `RichTextEditor` (`NSViewRepresentable` wrapping `NSTextView`):

- Configured for rich text editing
- Supports `NSTextAttachment` for inline images

- Images display at reasonable max width within editor
- Syncs `attributedString` (Data) and `content` (String) on the Prompt model
- Non-image files cannot be inlined — attachments strip only

Attachments Strip

Horizontal scrolling strip below the editor:

- Thumbnails for images/video, icon placeholders for audio/office files
- Add button opens `NSOpenPanel` file picker
- Drag-and-drop to reorder or accept files from Finder
- Right-click context menu: Delete, Open, Quick Look, Insert Inline (images only)
- Images dropped directly into the editor auto-create inline attachments

Rich Clipboard

When copying a prompt:

- `NSPasteboard` receives multiple representations:
- **RTFD**: Attributed string with inline images
- **Plain text**: content string with template variables resolved
- **File promises**: For non-inline attachments
- Menu bar and hotkey launcher continue to copy plain text only

Impact on Existing Features

- **Template variables**: Parse `{{var}}` from plain content string — unaffected
- **Search**: Operates on plain content string — unchanged
- **Menu bar / hotkey**: Copy plain text — unchanged
- **Inspector**: No changes needed

Decisions Summary

| Decision | Choice |

|-----|-----|

| Creation UX | Separate window |

| Variable fields | Auto-expanding `NSTextView` |

| Rich media approach | `NSTextView` + `NSTextAttachment` |

| Storage | Hybrid: $\leq 10\text{MB}$ embedded, $>10\text{MB}$ referenced |

| Inline media | Images only; other types in attachments strip |

| Clipboard | Rich (RTFD + plain text + file promises) |

| Scope | Main window only (not menu bar or hotkey launcher) |